

```

{////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////}
///                                                                    ///
///      Unidad Ayudas desarrollada por Javier Blanes Bravo          ///
///      para el proyecto "Desarrollo de un Programa Informático      ///
///      como Medio Didáctico en la Enseñanza de la                 ///
///      Teoría de Campos"                                           ///
///      SEP-94                                                       ///
///                                                                    ///
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////}
///                                                                    ///
///      La TPU incluye 2 procedimientos para la edición de          ///
///      de ficheros de ayuda en modo gráfico                       ///
///                                                                    ///
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////}

```

UNIT Ayudas;

INTERFACE

USES Dos, Crt, Graph, Raton;

PROCEDURE Ayuda(Fichero:String);
 PROCEDURE Marco_ayuda(x1,y1,x2,y2:Word);

IMPLEMENTATION

PROCEDURE Marco_Ayuda(X1,Y1,X2,Y2:Word);

CONST

Relieve=5;

VAR

I:Word;

BEGIN

SetColor(Blue);

Rectangle(X1,Y1,X2,Y2);

SetFillStyle(1,White);

Bar(X1+1,Y1+1,X2-1,Y2-1);

SetColor(LightGray);

for i:=1 to Relieve do

Rectangle(X1+i,Y1+i,X2-i,Y2-i);

SetColor(DarkGray);

For i:=Relieve+1 to Relieve+(3) do

Rectangle(X1+i,Y1+i,X2-i,Y2-i);

for i:=1 to Relieve do

begin

Line(X1+i,Y2-i,X2-i,Y2-i);

Line(X2-i,Y2-i,X2-i,Y1+i);

end;

SetColor(LightGray);

For I:=Relieve+1 to Relieve+(3) do

begin

Line(X1+i,Y2-i,X2-i,Y2-i);

Line(X2-i,Y2-i,X2-i,Y1+i);

end;

SetColor(1);

Rectangle(X1+11,Y1+13,X2-11,Y2-13);

Rectangle(X1+13,Y1+11,X2-13,Y2-11);

END;

PROCEDURE Ayuda(Fichero:String);

VAR

P,Q:Pointer;

Talla:Integer;

F:Text;

Ch:Char;

```

I, J: Word;
Fin: Boolean;
BEGIN
  No_Cursor;
  Talla:=ImageSize(100,100,540,275);
  GetMem(P,Talla);
  GetMem(Q,Talla);
  GetImage(100,100,540,275,P^);
  GetImage(100,275,540,450,Q^);
  Marco_Ayuda(100,100,540,450);
  SetViewPort(114,114,526,436,True);
  I:=10;
  J:=10;
  Assign(F,Fichero);
  SetColor(Blue);
  Reset(F);
  while not eof(f) do
    begin
      read(f,Ch);
      If eoln(F) then
        begin
          outtextxy(I, J, ch);
          J:=J+15;
          I:=10;
          read(F, Ch);
          read(F, Ch);
          read(F, ch);
        end;
      if ord(ch)=13 then
        begin
          J:=J+15;
          I:=10;
          read(f, ch);
          read(f, ch);
        end;
      If Ord(Ch)=36
      Then
        Begin
          If GetColor=Blue Then SetColor(Red)
          Else SetColor(Blue);
          Read(F, Ch);
        End;
      if not Eof(f)
      Then If GetColor=Blue Then outtextxy(I, J, Ch)
      Else
        Begin
          SetColor(8);
          OutTextXy(I+1, J+1, Ch);
          SetColor(Red);
          outtextxy(I, J, Ch);
        End;
      I:=I+10;
      if (J>280)
      Then
        Begin
          Ch:=ReadKey;
          J:=10;
          SetFillStyle(1,White);
          Bar(0,0,440,350);
          if ord(Ch)=71
          Then reset(F);
        End;
      If Eof(F)
      Then

```

```

begin
    Ch:=ReadKey;
    If Ord(Ch)<>27
    Then
        Begin
            J:=10;
            SetFillStyle(1,White);
            Bar(0,0,440,350);
            Reset(F);
        End;
    End;
end;
close(f);
SetViewPort(0,0,GetMaxX,GetMaxY,True);
PutImage(100,100,P^,NormalPut);
FreeMem(P,Talla);
PutImage(100,275,Q^,NormalPut);
FreeMem(Q,Talla);
Cursor;
END;
END.

```

```
{////////// FIN DE LA TPU AYUDAS //////////}
```

```

{//////////////////////////////////////
///
///      Unidad Cargas desarrollada por Javier Blanes Bravo      ///
///      para el proyecto "Desarrollo de un Programa Informático  ///
///      como Medio Didáctico en la Enseñanza de la              ///
///      Teoría de Campos"                                         ///
///      SEP-94                                                     ///
///                                                                ///
//////////////////////////////////////
///
///      La TPU incluye 5 procedimientos y funciones, y los      ///
///      objetos hilo y cargas para                               ///
///      el manejo de los conceptos teóricos asociados a la      ///
///      Teoría de Campos.                                         ///
///                                                                ///
//////////////////////////////////////}
```

```
UNIT Cargas;
```

```
INTERFACE
```

```
USES Dos,Crt,Graph;
```

```
CONST
```

```
    Radio=0.3;
```

```
TYPE
```

```
    Paleta_Carga=Record
```

```
        Carga_Mas,
        Carga_Menos,
        Línea_Mas,
        Línea_Menos:Byte;
```

```
    end;
```

```
    Pto=Record
```

```
        X,Y:Real;
```

```
    End;
```

```
{////////// OBJETO CARGA //////////}
```

Carga=Object

```
Centro:Pto;
Masa:Integer;
Positiva:Boolean;
Car_Mas,Car_Menos,Lin_Mas,Lin_Menos:Byte;

PROCEDURE Ini_Carga(X,Y:Real;Mas:integer);
PROCEDURE Ini_Colores(P:Paleta_Carga);
PROCEDURE Dibujar;
PROCEDURE Borrar(Col:Byte);
PROCEDURE Campo(Var E:pto;Mp:pto);
FUNCTION Potencial(Mp:Pto):Real;
FUNCTION Proximo(P:Pto):Boolean;
```

End;

{////////// OBJETO HILO //////////}

Hilo=Object

```
Centro,L1,L2:Pto;
Masa:Integer;
Positiva,Vertical:Boolean;
Car_Mas,Car_Menos,Lin_Mas,Lin_Menos:Byte;

PROCEDURE Ini_Hilo(X1,Y1,X2,Y2:Real;Mas:integer);
PROCEDURE Ini_Colores(P:Paleta_Carga);
PROCEDURE Dibujar;
PROCEDURE Borrar(Col:Byte);
PROCEDURE Campo(Var E:pto;Mp:pto);
FUNCTION Potencial(VAR Mp:Pto):Real;
FUNCTION Proximo(P:Pto):Boolean;
```

End;

{////////// PROCEDIMIENTOS VARIOS //////////}

```
PROCEDURE Suma_Vectores(Var V1:pto;V2:pto);
PROCEDURE Pto_Ini_Carga(O:Pto;N,Total:Byte;Var V:pto);
PROCEDURE Pto_Ini_Hilo(L1,L2:Pto;N,Total:Byte;Ver:Boolean;Var V1,V2:pto);
PROCEDURE Cambiar_Signo(Var V:Pto);
FUNCTION Ptos_Proximos(P,Q:Pto):Boolean;
```

IMPLEMENTATION

{////////// IMPLEMENTACION PROCEDIMIENTOS VARIOS //////////}

PROCEDURE Suma_Vectores(Var V1:Pto;V2:Pto);

BEGIN

V1.X:=V1.X+V2.X;

V1.Y:=V1.Y+V2.Y;

END;

PROCEDURE Pto_Ini_Carga(O:Pto;N,Total:Byte;Var V:Pto);

CONST

Pi=3.141593;

VAR

Alfa:Real;

BEGIN

Alfa:=(N*2*Pi)/Total;

V.X:=O.X+(Radio*Cos(Alfa));

V.Y:=O.Y+(Radio*Sin(Alfa));

END;

PROCEDURE Pto_Ini_Hilo(L1,L2:Pto;N,Total:Byte;Ver:Boolean;Var V1,V2:pto);

VAR

Dist:Real;

```

BEGIN
  If Ver
    Then
      begin
        Dist:=(L2.Y+0.2)-(L1.Y-0.2);
        V1.Y:=L1.Y-0.2+((Dist/(Total-1))*(N-1));
        V1.X:=L1.X+0.2;
        V2.Y:=V1.Y;
        V2.X:=L1.X-0.2;
      End
    Else
      begin
        Dist:=(L2.X+0.2)-(L1.X-0.2);
        V1.X:=L1.X-0.2+((Dist/(Total-1))*(N-1));
        V1.Y:=L1.Y+0.2;
        V2.X:=V1.X;
        V2.Y:=L1.Y-0.2;
      End;
    End;
END;

PROCEDURE Cambiar_Signo(Var V:Pto);
BEGIN
  V.X:=-V.X;
  V.Y:=-V.Y;
END;

FUNCTION Ptos_Proximos(P,Q:Pto):Boolean;
BEGIN
  Ptos_Proximos:=(Abs(P.X-Q.X)<0.1)and(Abs(P.Y-Q.Y)<0.1);
END;

{////////// IMPLEMENTACION OBJETO CARGA //////////}

PROCEDURE Carga.Ini_Carga(X,Y:Real;Mas:integer);
BEGIN
  Centro.X:=X;
  Centro.Y:=Y;
  Masa:=Mas;
  if Mas>0 then Positiva:=True
    else Positiva:=False;
END;

PROCEDURE Carga.Ini_Colores(P:Paleta_Carga);
BEGIN
  Car_Mas:=P.Carga_Mas;
  Car_Menos:=P.Carga_Menos;
  Lin_Mas:=P.Linea_Mas;
  Lin_Menos:=P.Linea_Menos;
END;

PROCEDURE Carga.Dibujar;
VAR
  I:Byte;
  X,Y:Word;
  T:String;
BEGIN
  If Positiva Then SetColor(Car_Mas)
    else SetColor(Car_Menos);
  X:=round(centro.x*100);
  Y:=round(centro.y*100);
  SetFillStyle(1,LightGray);
  Bar(X-13,Y-13,X+13,Y+13);
  For I:=0 to 5 do Circle(X,Y,18-I);
  SetTextJustify(1,1);

```

```

    Str(Masa,T);
    SetColor(0);
    OutTextXY(X,Y,T);
    SetTextJustify(0,2);
END;

```

```

PROCEDURE Carga.Borrar(Col:Byte);
VAR
    X,Y:Word;
BEGIN
    X:=round(centro.x*100);
    Y:=round(centro.y*100);
    SetFillStyle(1,Col);
    Bar(X-19,Y-19,X+19,Y+19);
END;

```

```

FUNCTION Carga.Proximo(P:Pto):Boolean;
VAR
    Dist,Cc,Co:Real;
BEGIN
    Cc:=P.X-Centro.X;
    Co:=P.Y-Centro.Y;
    Dist:=Sqrt(Sqr(Cc)+Sqr(Co));
    If (Dist<=Radio)
        Then Proximo:=True
        Else Proximo:=False;
END;

```

```

PROCEDURE Carga.Campo(Var E:Pto;Mp:Pto);
VAR
    Dist,Cc,Co:Real;
BEGIN
    Cc:=Mp.X-Centro.X;
    Co:=Mp.Y-Centro.Y;
    Dist:=Sqrt(Sqr(Cc)+Sqr(Co));
    E.X:=0.1*Masa*Cc/(Dist*Dist*Dist);
    E.Y:=0.1*Masa*Co/(Dist*Dist*Dist);
END;

```

```

FUNCTION Carga.Potencial(Mp:Pto):Real;
VAR
    Dist,Cc,Co:Real;
BEGIN
    Cc:=Mp.X-Centro.X;
    Co:=Mp.Y-Centro.Y;
    Dist:=Sqrt(Sqr(Cc)+Sqr(Co));
    If Dist=0 Then Potencial:=0
        Else Potencial:=Masa/Dist;
END;

```

```

{////////// IMPLEMENTACION OBJETO HILO //////////}

```

```

PROCEDURE Hilo.Ini_Hilo(X1,Y1,X2,Y2:Real;Mas:integer);
BEGIN
    Centro.X:=X1+((X2-X1)/2);
    Centro.Y:=Y1+((Y2-Y1)/2);
    L1.X:=X1;
    L1.Y:=Y1;
    L2.X:=X2;
    L2.Y:=Y2;
    Masa:=Mas;
    if L1.X=L2.X Then Vertical:=True
        Else Vertical:=False;
    if Mas>0 then Positiva:=True

```

```

        else Positiva:=False;
END;

PROCEDURE Hilo.Ini_Colores(P:Paleta_Carga);
BEGIN
    Car_Mas:=P.Carga_Mas;
    Car_Menos:=P.Carga_Menos;
    Lin_Mas:=P.Linea_Mas;
    Lin_Menos:=P.Linea_Menos;
END;

PROCEDURE Hilo.Dibujar;
VAR
    X1,Y1,X2,Y2:Word;
    T:String;
BEGIN
    If Positiva Then SetFillStyle(1,Car_Mas)
                    else SetFillStyle(1,Car_Menos);
    X1:=Round(L1.X*100);
    Y1:=Round(L1.Y*100);
    X2:=Round(L2.X*100);
    Y2:=Round(L2.Y*100);
    If L1.X=L2.X
        Then Bar(X1-4,Y1,X2+4,Y2)
        Else Bar(X1,Y1-4,X2,Y2+4);
    SetTextJustify(1,1);
    Str(Masa,T);
    SetColor(0);
    OutTextXY(Round(Centro.X*100),Round(Centro.Y*100),T);
    SetTextJustify(0,2);
END;

PROCEDURE Hilo.Borrar(Col:Byte);
VAR
    X1,Y1,X2,Y2:Word;
BEGIN
    SetFillStyle(1,Col);
    X1:=Round(L1.X*100);
    Y1:=Round(L1.Y*100);
    X2:=Round(L2.X*100);
    Y2:=Round(L2.Y*100);
    If L1.X=L2.X
        Then Bar(X1-12,Y1,X2+12,Y2)
        Else Bar(X1,Y1-12,X2,Y2+12);
END;

FUNCTION Hilo.Proximo(P:Pto):Boolean;
VAR
    Dist,Cc,Co:Real;
BEGIN
    If (P.X>L1.X-0.2)and(P.X<L2.X+0.2)and(P.Y>L1.Y-0.2)and(P.Y<L2.Y+0.2)
        Then Proximo:=True
        Else Proximo:=False;
END;

PROCEDURE Hilo.Campo(Var E:Pto;Mp:Pto);
VAR
    R1,R2,D,D1,D2:Real;
BEGIN
    If Vertical
        Then
            Begin
                D:=Mp.X-Centro.X;
                D1:=Mp.Y-L1.Y;
            End
        End
    End

```



```

///
/////////////////////////////////////////////////////////////////
///
///      El programa incluye gran nº de procedimientos      ///
///      y funciones que hacen uso de las librerías          ///
///      predefinidas y no que se especifican en Use.        ///
///                                                           ///
/////////////////////////////////////////////////////////////////}

```

PROGRAM Principal;

USES Dos,Crt,Graph,Grafico,Menus,Raton,Cargas,Pantalla,Ayudas,Sonido;

CONST

```

    Max_List=9;
    NP:Array[1..4] of String=      ('Opciones',
                                    'Parametros',
                                    'Colores',
                                    'Ayuda');

    LP:Array[1..4] of Char=        ('O','P','C','A');
    ND1:Array[1..9] of String=     ('Lineas_Campo',
                                    'Superficies_Equi',
                                    'Situar_Cargas',
                                    'Situar_Hilo',
                                    'Borrar_Cargas',
                                    'Borrar_Hilo',
                                    'Modificar_Cargas',
                                    'Modificar_Hilo',
                                    'Fin');

    LD1:Array[1..9] of Char=       ('L','S','C','H','B','r','M','o','F');
    ND2:Array[1..3] of String=     ('Densidad_De_Lineas',
                                    'Rapidez',
                                    'Grosor_De_Lineas');

    LD2:Array[1..3] of Char=       ('D','R','G');
    ND3:Array[1..8] of String=     ('Fondo',
                                    'Texto',
                                    'Letra',
                                    'Resalte',
                                    'Carga_+',
                                    'Carga_- ',
                                    'Lineas_+',
                                    'Lineas_- ');

    LD3:Array[1..8] of Char=       ('o','e','t','s','+','-','i','n');
    ND4:Array[1..4] of String=     ('De_Opciones',
                                    'De_Parametros',
                                    'De_Colores',
                                    'De_Ayuda');

    LD4:Array[1..4] of Char=       ('O','P','C','A');

```

TYPE

```

    Lista_Cargas=Array[0..Max_List] of Carga;
    Lista_Hilos=Array[0..Max_list] of Hilo;

```

VAR

```

    Llave,Fin           :Boolean;
    ch                   :char;
    Grosor,Densidad,Opcion,i,j :Integer;
    x1,y1,x2,y2,x3,y3,x4,y4 :Word;
    Pull                 :Ventana;
    Down                 :array[1..4] of Ventana;
    Pm,M                 :Paleta_Menu;
    Pc,C                 :Paleta_Carga;
    T                    :String;
    F                    :File of Paleta_Menu;

```

```

G                :File of Paleta_Carga;
L1               :Lista_Cargas;
L2               :Lista_Hilos;
N_Cargas,N_Hilos,Accion,Col:Byte;
Fac_Esc          :Real;

```

```
PROCEDURE Primera_Pantalla;
```

```
VAR
```

```
  I,J:Byte;
```

```
  Ch:Char;
```

```
BEGIN
```

```
  Bip2;
```

```
  SetFillstyle(1,White);
```

```
  For J:=0 to 9 do
```

```
    For I:=0 to 10 do
```

```
      Begin
```

```
        Bar(I*64,J*46,(i*64)+64,(j*46)+46);
```

```
        Delay(15);
```

```
      End;
```

```
  SetColor(Blue);
```

```
  Rectangle(20,30,620,430);
```

```
  Rectangle(30,20,610,440);
```

```
  Setcolor(Blue);
```

```
  SetTextJustify(1,1);
```

```
  OutTextXY(323,200,'Programa realizado por Javier Blanes Bravo');
```

```
  OutTextXY(323,220,'el 15 de Septiembre de 1994');
```

```
  OutTextXY(323,240,'como Proyecto Fin de Carrera');
```

```
  OutTextXY(323,260,'bajo el titulo:');
```

```
  OutTextXY(323,280,'"Desarrollo de un Programa Informático');
```

```
  OutTextXY(323,300,'Como Medio Didáctico en la Enseñanza de la');
```

```
  OutTextXY(323,320,'Teoría de Campos",'');
```

```
  OutTextXY(323,340,'para el departamento de Física Aplicada de la E.U.I.');
```

```
  OutTextXY(323,360,'de la Universidad Politécnica de Valencia.');
```

```
  SetColor(Red);
```

```
  OutTextXY(323,400,'PRESIONE ENTER PARA CONTINUAR');
```

```
  SetColor(7);
```

```
  SetTextStyle(3,0,4);
```

```
  OutTextXY(323,103,'PROGRAMA CARGAS E HILOS');
```

```
  SetColor(Red);
```

```
  OutTextXY(320,100,'PROGRAMA CARGAS E HILOS');
```

```
  Repeat Until KeyPressed;
```

```
  Ch:=ReadKey;
```

```
  SetTextStyle(0,0,0);
```

```
  SetTextJustify(0,2);
```

```
  Bip2;
```

```
END;
```

```
{///// DETECTA SI HA EXISTIDO PULSACION DE TECLADO O RATON /////}
```

```
FUNCTION Evento:Boolean;
```

```
BEGIN
```

```
  If (KeyPressed)or(Botones<>0) Then Evento:=True
```

```
    Else Evento:=False;
```

```
END;
```

```
{///// APLICA UN FACTOR DE ESCALA A LA VARIABLE DE TIPO VECTOR /////}
```

```
PROCEDURE Factor_Esc(Var P:pto;Nulo:Boolean);
```

```
VAR
```

```
  C:Real;
```

```
BEGIN
```

```
  if Abs(P.X)>Abs(P.Y) Then C:=Fac_Esc/Abs(P.X)
```

```
    Else C:=Fac_Esc/Abs(P.Y);
```

```
  P.X:=C*P.X;
```

```

P.Y:=C*P.Y;
if c>1 then Nulo:=true
      else Nulo:=false;
END;

{///// ESTABLECE UN VALOR ENTRE DOS LIMITES  /////}

PROCEDURE Establecer_Un_Valo(r VAR V:Integer;Lim_Sup,Lim_Inf:Integer);
VAR
  Ca:Caja;
  Di:Display;
  Ok,Izda,Dcha:Boton;
  Salir:Boolean;
  T1,T2:String;
BEGIN
  Ca.Ini_Caja(190,40,445,200,3);
  No_Cursor;
  Ca.Ver;
  Texto(317,135,Blue,'Etablecer el valor del ');
  Texto(317,150,Blue,'parametro entre los limites:');
  Str(Lim_Inf,T1);
  Str(Lim_Sup,T2);
  Texto(317,165,Blue,'- Inferior: ');
  Texto(360,165,Red,T1);
  Texto(317,180,Blue,'- Superior: ');
  Texto(360,180,Red,T2);
  Di.Ini_Display(260,60,V);
  Izda.Ini_Boton(330,60,350,80,chr(17),3,0);
  Dcha.Ini_Boton(355,60,375,80,chr(16),3,0);
  Ok.Ini_Boton(270,100,365,120,'Ok',3,0);
  Izda.Ver;
  Dcha.Ver;
  Ok.Ver;
  Cursor;
  Salir:=False;
  Repeat
    if Dcha.Esta_pulsado then
      Begin
        If V<Lim_Sup
          Then
            begin
              V:=V+1;
              Di.Incremento(1);
            end;
            Dcha.Ver_Pulsado;
            Delay(200);
            Dcha.Ver;
          end;
        if Izda.Esta_pulsado then
          begin
            If V>Lim_Inf
              Then
                begin
                  V:=V-1;
                  Di.Incremento(-1);
                end;
                Izda.Ver_Pulsado;
                Delay(200);
                Izda.Ver;
              end;
            if Ok.Esta_Pulsado
              then
                begin
                  Salir:=True;

```

```

                Ok.Ver_Pulsado;
                Delay(300);
                Ok.Ver;
            end;
        Until Salir;
        Ca.Borrar;
    END;

{///// DIBUJA LAS LINEAS DE CAMPO PARA LAS CARGAS  /////}

PROCEDURE Linea_Carga(I:Byte);
VAR
    E1,E2:Array[0..Max_List] of Pto;
    P:Pto;
    J,K:Byte;
    Proximo:Boolean;
BEGIN
    For J:=1 to Densidad do
        Begin
            Pto_Ini_Carga(L1[i].Centro,J,Densidad,P);
            Proximo:=False;
            Repeat
                For K:=0 to N_Cargas-1 do L1[k].Campo(E1[k],P);
                If N_Hilos>0 Then For K:=0 to N_Hilos-1 do
                    L2[k].Campo(E2[k],P);
                For K:=0 to N_Cargas-1 do
                    If I<>K Then Suma_Vectores(E1[i],E1[k]);
                If N_Hilos>0 Then For K:=0 to N_Hilos-1 do
                    Suma_Vectores(E1[i],E2[k]);
                If Not(L1[I].Positiva) Then cambiar_Signo(E1[i]);
                Factor_Esc(E1[i],proximo);
                Suma_Vectores(P,E1[i]);
                If L1[I].Positiva
                    Then Dibuja_Pto(P,Pc.Linea_Mas,Grosor)
                    Else Dibuja_Pto(P,Pc.Linea_Menos,Grosor);
                For K:=0 to N_Cargas-1 do
                    Proximo:=(Proximo)or(L1[K].Proximo(P));
                If N_Hilos>0
                    Then For K:=0 to N_Hilos-1 do
                        Proximo:=(Proximo)or(L2[k].Proximo(P));
                Until Fuera_pan(P)or(Proximo);
            End;
        End;
    END;

{///// DIBUJA LAS LINEAS DE CAMPO PARA LOS HILOS  /////}

PROCEDURE Linea_Hilo(I:Byte);
VAR
    E1,E2:Array[0..Max_List] of Pto;
    P1,P2:Pto;
    J,K:Byte;
    Proximo:Boolean;
BEGIN
    For J:=1 to Densidad do
        Begin
            Pto_Ini_Hilo(L2[i].L1,L2[i].L2,J,Densidad,
                L2[i].Vertical,P1,P2);
            Proximo:=False;
            Repeat
                If N_Cargas>0 Then For K:=0 to N_Cargas-1 do
                    L1[k].Campo(E1[k],P1);
                For k:=0 to N_Hilos-1 do L2[k].Campo(E2[k],P1);
                If N_Cargas>0 Then For K:=0 to N_Cargas-1 do

```

```

        Suma_Vectores(E2[i],E1[k]);
    For K:=0 to N_Hilos-1 do
        If I<>K Then Suma_Vectores(E2[i],E2[k]);
    If not(L2[I].Positiva) Then cambiar_Signo(E2[i]);
    Factor_Esc(E2[i],Proximo);
    Suma_Vectores(P1,E2[i]);
    If L2[I].Positiva
        Then Dibuja_Pto(P1,Pc.Linea_Mas,Grosor)
        Else Dibuja_Pto(P1,Pc.Linea_Menos,Grosor);
    If N_Cargas>0 Then For k:=0 to N_Cargas-1 do
        Proximo:=(Proximo)or(L1[K].Proximo(P1));
    For K:=0 to N_Hilos-1 do
        Proximo:=(Proximo)or(L2[k].Proximo(P1));
    Until Fuera_pan(P1)or(Proximo);
    Pto_Ini_Hilo(L2[i].L1,L2[i].L2,J,Densidad,
        L2[i].Vertical,P1,P2);
    Proximo:=False;
    Repeat
        If N_Cargas>0 Then For K:=0 to N_Cargas-1 do
            L1[k].Campo(E1[k],P2);
        For k:=0 to N_Hilos-1 do L2[k].Campo(E2[k],P2);
        If N_Cargas>0 Then For K:=0 to N_Cargas-1 do
            Suma_Vectores(E2[i],E1[k]);
        For K:=0 to N_Hilos-1 do
            If I<>K Then Suma_Vectores(E2[i],E2[k]);
        If not(L2[I].Positiva) Then cambiar_Signo(E2[i]);
        Factor_Esc(E2[i],Proximo);
        Suma_Vectores(P2,E2[i]);
        If L2[I].Positiva
            Then Dibuja_Pto(P2,Pc.Linea_Mas,Grosor)
            Else Dibuja_Pto(P2,Pc.Linea_Menos,Grosor);
        IF N_Cargas>0 Then For k:=0 to N_Cargas-1 do
            Proximo:=(Proximo)or(L1[K].Proximo(P2));
        For k:=0 to N_Hilos do
            Proximo:=(Proximo)or(L2[k].Proximo(P2));
        Until Fuera_pan(P2)or(Proximo);
    End;

END;

{///// DIBUJA LAS SUPERFICIES EQUIPOTENCIALES /////}

PROCEDURE Equi_Pot;
VAR
    i,j,k,Color,m,n:Word;
    pot,Dist:Real;
    Max,Min:Integer;
    p:pto;
    prox:Boolean;
PROCEDURE Cuadro(I,J,C:Byte);
BEGIN
    Setcolor(c);
    rectangle(I*4-1,J*4-1,I*4+1,J*4+1);
    putpixel(i*4,j*4,c);
END;
BEGIN
    Max:=0;
    If N_Cargas>0 Then for k:=0 to N_Cargas-1 do
        If Abs(L1[K].Masa)>Max Then Max:=Abs(L1[K].Masa);
    If N_Hilos>0 Then for k:=0 to N_Hilos-1 do
        If Abs(L2[K].Masa)>Max Then Max:=Abs(L2[K].Masa);
    i:=0;
    j:=4;
    while i<161 do

```

```

Begin
    p.x:=i*4/100;
    p.y:=j*4/100;
    prox:=false;
    If N_Cargas>0 Then for k:=0 to N_Cargas-1 do
        prox:=prox or l1[k].proximo(p);
    If N_Hilos>0 Then for k:=0 to N_Hilos-1 do
        prox:=prox or l2[k].proximo(p);
    if not(prox) Then
        begin
            pot:=0;
            If N_Cargas>0 Then for k:=0 to N_Cargas-1 do
                pot:=pot+L1[k].Potencial(p);
            If N_Hilos>0 Then for k:=0 to N_Hilos-1 do
                pot:=pot+L2[k].Potencial(p);
            Pot:=pot*3/Max;
            Color:=round(pot);
            If Color=Cyan Then Color:=0;
            for m:=0 to grosor do
                for n:=0 to grosor do
                    cuadro(i+m,j+n,Color);
            end;
            j:=j+grosor+1;
            if j>120 then
                begin
                    j:=4;
                    i:=i+grosor+1;
                end;
            end;
        End;
    END;

{///// SITUA LAS CARGAS EN PANTALLA /////}

PROCEDURE Situar_Cargas;
VAR
    X,Y:Word;
    Ca,Ca1,Ra:Caja;
    Di:Display;
    Izda,Dcha,Ok,Fin,Otra:Boton;
    Val:Integer;
    Salir:Boolean;
    Desp:Byte;
BEGIN
    Ra.Ini_Caja(489,0,639,40,3);
    Ra.Ver;
    Ra.Ver_Posicion_Raton;
    Ca1.Ini_Caja(60,458,579,478,1);
    Ca1.Ver;
    Texto(240,468,Blue,'- Situar la carga con :');
    Texto(400,468,Red,'Ratón_Enter');
    Repeat
        If (Ra.cor_X<>Donde_X)or(Ra.Cor_Y<>Donde_Y)
            Then Ra.Actualizar_Raton;
        Delay(100);
    Until (Botones<>0)and(Dentro_Region(20,35,GetMaxX-20,GetMaxY-20));
    Ra.Borrar;
    Ca1.Borrar;
    X:=Donde_X;
    Y:=Donde_Y;
    If Dentro_Region(160,0,475,240) Then Desp:=240
        Else Desp:=0;
    No_Cursor;
    Ca.Marca(X,Y);
    Cursor;

```

```

Ca.Ini_Caja(190,20+Desp,445,220+Desp,3);
Ca.Ver;
Texto(312,150+Desp,Blue,'Establecer el valor de la');
Texto(312,165+Desp,Blue,'carga (0 imposible) entre :');
Texto(312,180+Desp,Blue,'- Limite Inferior : ');
Texto(400,180+Desp,Red,'-40');
Texto(312,195+Desp,Blue,'- Limite Superior : ');
Texto(400,195+Desp,Red,'40');
Val:=0;
Di.Ini_Display(260,40+Desp,0);
Izda.Ini_Boton(330,40+Desp,350,60+Desp,chr(17),3,0);
Dcha.Ini_Boton(355,40+Desp,375,60+Desp,chr(16),3,0);
Ok.Ini_Boton(270,80+Desp,365,100+Desp,'Ok',3,0);
Fin.Ini_Boton(270,110+Desp,315,130+Desp,'Fin',3,0);
Otra.Ini_Boton(320,110+Desp,365,130+Desp,'Otra',3,0);
Izda.Ver;
Dcha.Ver;
Ok.Ver;
Fin.Ver;
Otra.Ver;
Salir:=False;
Repeat
    if Dcha.Esta_pulsado then
        Begin
            If Val <=40
                Then
                    Begin
                        Val:=Val+1;
                        Di.Incremento(1);
                    End;
                    Dcha.Ver_Pulsado;
                    Delay(200);
                    Dcha.Ver;
                end;
            if Izda.Esta_pulsado then
                begin
                    If Val >=-40
                        Then
                            Begin
                                Val:=Val-1;
                                Di.Incremento(-1);
                            End;
                            Izda.Ver_Pulsado;
                            Delay(200);
                            Izda.Ver;
                        end;
                    if (Ok.Esta_Pulsado)and(Val<>0)
                        then
                            begin
                                Salir:=True;
                                Ok.Ver_Pulsado;
                                Delay(300);
                                Ok.Ver;
                                L1[N_Cargas].Ini_Carga(X/100 ,Y/100,Val);
                                L1[N_Cargas].Ini_Colores(Pc);
                                L1[N_Cargas].Dibujar;
                                N_Cargas:=(N_Cargas+1);
                            end;
                end;
            until Salir;
            Salir:=False;
        repeat
            if Fin.esta_Pulsado
                then
                    begin

```

```

        Salir:=True;
        Fin.Ver_Pulsado;
        Delay(300);
        Fin.Ver;
        Delay(200);
    End;
    if Otra.Esta_Pulsado then
        begin
            Salir:=true;
            Otra.Ver_Pulsado;
            Delay(300);
            Otra.Ver;
            Ca.Borrar;
            situar_cargas;
        end;
    Until Salir;
    Ca.Borrar;
END;

{///// SITUA LOS HILOS EN PANTALLA /////}

PROCEDURE Situar_Hilos;
VAR
    X1,Y1,X2,Y2:Word;
    Ca,Ca1,Ca2,Ra:Caja;
    Di:Display;
    Izda,Dcha,Ok,Fin,Otra:Boton;
    Val:Integer;
    Salir:Boolean;
    Desp:Byte;
    Ch:Char;
BEGIN
    Ra.Ini_Caja(489,0,639,40,3);
    Ra.Ver;
    Ra.Ver_Posicion_Raton;
    Ca1.Ini_Caja(60,458,579,478,1);
    Ca1.Ver;
    Texto(240,468,Blue,'- Situar el hilo con :');
    Texto(400,468,Red,'Rat#n_Enter');
    Repeat
        If (Ra.cor_X<>Donde_X)or(Ra.Cor_Y<>Donde_Y)
            Then Ra.Actualizar_Raton;
        Delay(100);
    Until (Botones<>0)and(Dentro_Region(20,35,GetMaxX-20,GetMaxY-20));
    Ra.Borrar;
    Ca1.Borrar;
    X1:=Donde_X;
    Y1:=Donde_Y;
    Ca.Marca(X1,Y1);
    Ca2.Ini_Caja(60,458,579,478,1);
    Ca2.Ver;
    Texto(250,468,Blue,'- Hilo horizontal: - Vertical:');
    Texto(300,468,Red,'(H) o (h)');
    Texto(500,468,Red,'(V) o (v)');
    Repeat
        Ch:=ReadKey;
    Until (Ch='H')or(Ch='h')or(Ch='V')or(Ch='v');
    Ca2.Borrar;
    If (Ch='H')or(Ch='h')
        Then Limites(X1,Y1,GetMaxX,Y1)
        Else Limites(X1,Y1,X1,GetMaxy);
    Ra.Ini_Caja(489,0,639,40,3);
    Ra.Ver;
    Ra.Ver_Posicion_Raton;

```



```

Repeat
    If (Ra.cor_X<>Donde_X)or(Ra.Cor_Y<>Donde_Y)
        Then Ra.Actualizar_Raton;
        Delay(100);
Until (Botones<>0)and(Dentro_Region(20,35,GetMaxX-20,GetMaxY-20));
Ra.Borrar;
X2:=Donde_X;
Y2:=Donde_Y;
Limites(0,0,GetMaxX,GetMaxY);
If Dentro_Region(160,0,475,240) Then Desp:=240
    Else Desp:=0;

No_Cursor;
Ca.Marca(X2,Y2);
Cursor;
Ca.Ini_Caja(190,20+Desp,445,220+Desp,3);
Ca.Ver;
Texto(312,150+Desp,Blue,'Establecer el valor de la');
Texto(312,165+Desp,Blue,'carga (0 imposible) entre :');
Texto(312,180+Desp,Blue,'- Limite Inferior : ');
Texto(400,180+Desp,Red,'-50');
Texto(312,195+Desp,Blue,'- Limite Superior : ');
Texto(400,195+Desp,Red,'50');
Val:=0;
Di.Ini_Display(260,40+Desp,0);
Izda.Ini_Boton(330,40+Desp,350,60+Desp,chr(17),3,0);
Dcha.Ini_Boton(355,40+Desp,375,60+Desp,chr(16),3,0);
Ok.Ini_Boton(270,80+Desp,365,100+Desp,'Ok',3,0);
Fin.Ini_Boton(270,110+Desp,315,130+Desp,'Fin',3,0);
Otra.Ini_Boton(320,110+Desp,365,130+Desp,'Otro',3,0);
Izda.Ver;
Dcha.Ver;
Ok.Ver;
Fin.Ver;
Otra.Ver;
Salir:=False;
Repeat
    if Dcha.Esta_pulsado then
        Begin
            If Val <50
                Then
                    Begin
                        Val:=Val+1;
                        Di.Incremento(1);
                    End;
                    Dcha.Ver_Pulsado;
                    Delay(200);
                    Dcha.Ver;
                end;
            if Izda.Esta_pulsado then
                begin
                    If Val >-50
                        Then
                            Begin
                                Val:=Val-1;
                                Di.Incremento(-1);
                            End;
                            Izda.Ver_Pulsado;
                            Delay(200);
                            Izda.Ver;
                        end;
                    if (Ok.Esta_Pulsado)and(Val<>0)
                        then
                            begin
                                Salir:=True;

```

```

Ok.Ver_Pulsado;
Delay(300);
Ok.Ver;
L2[N_Hilos].Ini_Hilo(X1/100,Y1/100,
                    X2/100,Y2/100,Val);
L2[N_Hilos].Ini_Colores(Pc);
N_Hilos:=N_Hilos+1;
end;
Until Salir;
Salir:=False;
repeat
    if Fin.esta_Pulsado
    then
        begin
            Salir:=True;
            Fin.Ver_Pulsado;
            Delay(300);
            Fin.Ver;
            Delay(200);
            Ca.Borrar;
            SetColor(Cyan);
            No_Cursor;
            OutTextXY(X1,Y1,'û');
            OutTextXY(X2,Y2,'û');
            L2[N_Hilos-1].Dibujar;
            Cursor;
        End;
    if Otra.Esta_Pulsado then
        begin
            Salir:=true;
            Otra.Ver_Pulsado;
            Delay(300);
            Otra.Ver;
            Ca.Borrar;
            SetColor(Cyan);
            No_Cursor;
            OutTextXY(X1,Y1,'û');
            OutTextXY(X2,Y2,'û');
            L2[N_Hilos-1].Dibujar;
            Cursor;
            Situar_Hilos;
        end;
    Until Salir;
END;

{///// BORRA CARGAS DE PANTALLA /////}

PROCEDURE Borrar_Cargas;
VAR
    X,Y:Word;
    Exito:Boolean;
    I,J:Byte;
    P:Pto;
    Ca:Caja;
BEGIN
    Repeat
        Ca.Ini_Caja(60,458,579,478,1);
        Ca.Ver;
        Texto(250,468,Blue,'-Seleccionar carga : -Salir :');
        Texto(300,468,Red,'Ratñ_Enter');
        Texto(500,468,Red,'Ratñ_Escape');
        Repeat Until (Botones<>0);
        X:=Donde_X;
        Y:=Donde_Y;

```

```

        Exito:=False;
        I:=0;
        P.X:=X/100;
        P.Y:=Y/100;
        While (I<=N_Cargas-1)and not(Exito) do
            If L1[I].Proximo(P) Then Exito:=True
                Else I:=I+1;
        If Exito Then
            Begin
                No_Cursor;
                L1[I].Borrar(3);
                Cursor;
                N_Cargas:=N_Cargas-1;
                For J:=I+1 to N_Cargas do
                    Begin
                        L1[J-1].Ini_Carga(L1[J].Centro.X,L1[J].Centro.Y,
                            L1[J].Masa);
                        L1[J-1].Ini_Colores(Pc);
                    end;
                End;
            Ca.Borrar;
            Until (Botones=2);
        END;

{///// BORRA HILOS DE PANTALLA /////}

PROCEDURE Borrar_Hilos;
VAR
    X,Y:Word;
    Exito:Boolean;
    I,J:Byte;
    P:Pto;
    Ca:Caja;
BEGIN
    Repeat
        Ca.Ini_Caja(60,458,579,478,1);
        Ca.Ver;
        Texto(250,468,Blue,'-Seleccionar hilo :           -Salir :');
        Texto(300,468,Red,'Ratñ_Enter');
        Texto(500,468,Red,'Ratñ_Escape');
        Repeat Until (Botones<>0);
        X:=Donde_X;
        Y:=Donde_Y;
        Exito:=False;
        I:=0;
        P.X:=X/100;
        P.Y:=Y/100;
        While (I<=N_Hilos-1)and not(Exito) do
            If L2[I].Proximo(P) Then Exito:=True
                Else I:=I+1;
        If Exito Then
            Begin
                No_Cursor;
                L2[I].Borrar(3);
                Cursor;
                N_Hilos:=N_Hilos-1;
                For J:=I+1 to N_Hilos do
                    Begin
                        L2[J-1].Ini_Hilo(L2[J].L1.X,L2[J].L1.Y,
                            L2[J].L2.X,L2[J].L2.Y,L2[J].Masa);
                        L2[J-1].Ini_Colores(Pc);
                    end;
                End;
            Ca.Borrar;

```

```

    Until (Botones=2);
END;

{///// MODIFICA EL VALOR DE CARGAS EN PANTALLA /////}

PROCEDURE Modificar_Cargas;
VAR
    X,Y:Word;
    Exito:Boolean;
    I,J:Byte;
    P:Pto;
    Val:Integer;
    Ca:Caja;
BEGIN
    Repeat
        Ca.Ini_Caja(60,458,579,478,1);
        Ca.Ver;
        Texto(260,468,Blue,'- Seleccionar carga :           y salir :');
        Texto(310,468,Red,'Ratón_Enter');
        Texto(505,468,Red,'Ratón_Escape');
        Repeat Until (Botones<>0);
        Ca.Borrar;
        X:=Donde_X;
        Y:=Donde_Y;
        Exito:=False;
        I:=0;
        P.X:=X/100;
        P.Y:=Y/100;
        While (I<=N_Cargas-1)and not(Exito) do
            If L1[I].Proximo(P) Then Exito:=True
                Else I:=I+1;
        If Exito Then
            Begin
                Val:=L1[I].Masa;
                Establecer_Un_Valor(Val,50,-50);
                No_Cursor;
                L1[I].Borrar(3);
                L1[I].Ini_Carga(L1[i].Centro.X,L1[i].Centro.Y,Val);
                L1[i].Dibujar;
                Cursor;
            End;
        Until (Botones=2);
    END;

```

```

{///// MODIFICA EL VALOR DE HILOS EN PANTALLA /////}

```

```

PROCEDURE Modificar_Hilos;
VAR
    X,Y:Word;
    Exito:Boolean;
    I,J:Byte;
    P:Pto;
    Val:Integer;
    Ca:Caja;
BEGIN
    Repeat
        Ca.Ini_Caja(60,458,579,478,1);
        Ca.Ver;
        Texto(260,468,Blue,'- Seleccionar hilo :           y salir :');
        Texto(310,468,Red,'Ratón_Enter');
        Texto(505,468,Red,'Ratón_Escape');
        Repeat Until (Botones<>0);
        Ca.Borrar;
        X:=Donde_X;

```

```

        Y:=Donde_Y;
        Exito:=False;
        I:=0;
        P.X:=X/100;
        P.Y:=Y/100;
        While (I<=N_Hilos-1)and not(Exito) do
            If L2[I].Proximo(P) Then Exito:=True
                Else I:=I+1;
        If Exito Then
            Begin
                Val:=L2[I].Masa;
                Establecer_Un_Valor(Val,50,-50);
                No_Cursor;
                L2[I].Borrar(3);
                L2[I].Ini_Hilo(L2[i].L1.X,L2[i].L1.Y,
                    L2[i].L2.X,L2[i].L2.Y,Val);
                L2[i].Dibujar;
                Cursor;
            End;
        Until (Botones=2);
    END;

{///// VISUALIZA TODAS LAS CARGAS EN PANTALLA /////}

PROCEDURE Ver_Todas_Cargas;
VAR
    I:Byte;
BEGIN
    If N_Cargas<>0 Then For I:=0 to N_Cargas-1 do L1[I].Dibujar;
END;

{///// VISUALIZA TODOS LOS HILOS EN PANTALLA /////}

PROCEDURE Ver_Todos_Hilos;
VAR
    I:Byte;
BEGIN
    If N_Hilos<>0 Then For I:=0 to N_Hilos-1 do L2[I].Dibujar;
END;

{///// OBTIENE EL CODIGO DE UN COLOR /////}

PROCEDURE Obtener_Un_Color(VAR Color:Byte);
VAR
    Ca:Caja;
    Dcha,Izda,Ok:Boton;
    Salir:Boolean;
    I:Byte;
BEGIN
    Ca.Ini_Caja(140,100,500,250,3);
    Ca.Ver;
    Texto(320,200,Blue,'Utilice los botones           para establecer');
    Texto(320,220,Blue,'el color deseado y           para terminar');
    Texto(330,200,Red,Chr(17));
    Texto(345,200,Red,Chr(16));
    Texto(340,220,Red,'Ok');
    Ca.Ver_Paleta(160,120);
    Ca.Marca(165+Color*15,145);
    dcha.Ini_Boton(435,120,455,140,chr(16),3,0);
    Izda.Ini_Boton(410,120,430,140,chr(17),3,0);
    Ok.Ini_Boton(300,160,340,180,' Ok',3,0);
    dcha.Ver;
    Izda.Ver;
    Ok.Ver;

```

```

Salir:=False;
Repeat
    if Dcha.Esta_Pulsado
    then
        begin
            Ca.No_Marca(165+Color*15,145);
            Color:=(Color+1)mod 16;
            Ca.Marca(165+Color*15,145);
            Dcha.Ver_Pulsado;
            Delay(300);
            Dcha.Ver;
        end;
    if Izda.Esta_Pulsado
    then
        begin
            Ca.No_Marca(165+Color*15,145);
            if Color=0 then Color:=15
            else Color:=(Color-1);
            if color<0 then color:=15;
            Ca.Marca(165+Color*15,145);
            Izda.Ver_Pulsado;
            Delay(300);
            Izda.Ver;
        end;
    if Ok.Esta_Pulsado
    then
        begin
            Salir:=True;
            OK.Ver_Pulsado;
            Delay(300);
        end;
    Until Salir;
    Ca.Borrar;
END;

```

```
{///// ESTABLECE LOS NUEVOS COLORES /////}
```

```

PROCEDURE Nuevos_Colores;
VAR
    I:Byte;
BEGIN
    Pull.Ini_Color(Pm);
    Pull.Borrar;
    Pull.Ver;
    For I:=1 to 4 do Down[I].Ini_Color(Pm);
End;

```

```
{///// LIMPIA LA PANTALLA /////}
```

```

PROCEDURE Borrar_Pantalla;
VAR
    I,J:Byte;
BEGIN
    No_Cursor;
    SetFillstyle(1,Cyan);
    For J:=0 to 10 do
        For I:=0 to 9 do
            Begin
                Bar(I*64,J*46+15,(i*64)+64,(j*46)+61);
                Delay(15);
            End;
        Cursor;
    END;

```

```
{///// GUARDA LA CONFIGURACION DE COLORES ACTUAL /////}
```

```
PROCEDURE Guardar_Configuracion;
```

```
VAR
```

```
Ca:Caja;  
Si,No:Boton;  
Sil,Nol:Boolean;  
Ch:Char;
```

```
BEGIN
```

```
Ca.Ini_Caja(160,120,455,240,3);  
Ca.Ver;  
Si.Ini_Boton(260,180,285,200,'Si',3,Black);  
No.Ini_Boton(350,180,375,200,'No',3,Black);  
Si.Ver;  
No.Ver;  
Texto(305,140,Blue,'¿ Desea guardar la configuraci#n');  
Texto(305,160,Blue,'de colores actual ? S/N :');  
Sil:=False;  
nol:=False;  
Repeat  
If Keypressed Then  
    Begin  
        Ch:=ReadKey;  
        Sil:=(Ch='S')or(Ch='s');  
        Nol:=(Ch='N')or(Ch='n');  
    end;  
until (Si.Esta_Pulsado)or(No.Esta_Pulsado)or(Sil)or(Nol);  
if Si.Esta_Pulsado Then  
    Begin  
        Si.Ver_Pulsado;  
        Delay(300);  
    end;  
if No.Esta_Pulsado Then  
    Begin  
        No.Ver_Pulsado;  
        Delay(300);  
    end;  
If (Si.Esta_Pulsado)or(Sil)  
Then  
    Begin  
        Assign(f,'color1.cnf');  
        Rewrite(F);  
        Write(F,Pm);  
        Close(F);  
        Assign(G,'color2.cnf');  
        Rewrite(G);  
        Write(G,Pc);  
        Close(G);  
    end;  
end;
```

```
END;
```

```
{///// PROCEDIMIENTO DE MANEJO DE MENUS /////}
```

```
PROCEDURE Menu_Down(Op:Byte);
```

```
VAR
```

```
E:Integer;  
I:Byte;
```

```
BEGIN
```

```
i:=Op;  
with Down[Op]do  
begin  
    ver;  
    Accion:=0;  
    repeat
```

```

repeat until evento;
E:=Tratar_Evento;
case E of
  1:begin
    Borrar;
    Pull.Resaltar_Op(I,True);
    if (I+1)<=Pull.Numero_Op
      then I:=I+1
      else I:=1;
    Pull.Resaltar_Op(I,False);
    Menu_Down(I)
  end;
  -1:begin
    borrar;
    Pull.Resaltar_Op(I,True);
    if (I-1)<>0
      then I:=I-1
      else I:=Pull.Numero_Op;
    Pull.Resaltar_Op(I,False);
    Menu_Down(I)
  end;
  27:Begin
    Borrar;
    Llave:=True;
  end;
  13:begin
    Borrar;
    Accion:=Obtener_Op;
  end;
end;
until (E<>0)
end;
END;

{///// PROCEDIMIENTO DE MANEJO DE MENUS /////}

PROCEDURE Menu;
VAR
  E:Integer;
  Fin:Boolean;
  K:Byte;
BEGIN
  Fin:=False;
  repeat
    repeat until (Evento);
    E:=Pull.Tratar_Evento;
    if (E=13) then Menu_Down(Pull.Obtener_Op);
    Case Accion of
      5:Begin
        Borrar_Pantalla;
        Ver_Todas_Cargas;
        Ver_Todos_Hilos;
        No_Cursor;
        If (N_Cargas<>0) Then For K:=0 to N_Cargas-1 do
          Linea_Carga(K);
        If (N_Hilos<>0) Then For K:=0 to N_Hilos-1 do
          Linea_Hilo(K);
        Cursor;
      End;
    6:Begin
      Borrar_Pantalla;
      Ver_Todas_Cargas;
      Ver_Todos_Hilos;
      No_Cursor;

```



```

        If (N_Cargas<>0)or(N_Hilos<>0)
            Then
                Begin
                    equi_pot;
                End;
            Cursor;
        End;
7:Situar_Cargas;
8:Situar_Hilos;
9:Borrar_Cargas;
10:Borrar_Hilos;
11:Modificar_Cargas;
12:Modificar_Hilos;
13:Fin:=True;
20:Establecer_Un_Valor(Densidad,40,2);
21:Begin
    I:=Round(Fac_Esc*100);
    Establecer_Un_Valor(i,3,1);
    Fac_Esc:=i/100;
end;
22:Establecer_Un_Valor(Grosor,1,0);
30:Begin
    Obtener_Un_Color(Pm.Fondo);
    Nuevos_Colores;
End;
31:Begin
    Obtener_Un_Color(Pm.Texto);
    Nuevos_Colores;
End;
32:Begin
    Obtener_Un_Color(Pm.Letra);
    Nuevos_Colores;
End;
33:Begin
    Obtener_Un_Color(Pm.Resalte);
    Nuevos_Colores;
End;
34:Begin
    Obtener_Un_Color(Pc.Carga_Mas);
    For I:=0 to N_Cargas-1 do L1[I].Ini_Colores(Pc);
End;
35:Begin
    Obtener_Un_Color(Pc.Carga_Menos);
    For I:=0 to N_Cargas-1 do L1[I].Ini_Colores(Pc);
End;
36:Begin
    Obtener_Un_Color(Pc.Linea_Mas);
    For I:=0 to N_Cargas-1 do L1[I].Ini_Colores(Pc);
End;
37:Begin
    Obtener_Un_Color(Pc.Linea_Menos);
    For I:=0 to N_Cargas-1 do L1[I].Ini_Colores(Pc);
End;
40:Ayuda('opciones.txt');
41:Ayuda('para.txt');
42:Ayuda('colores.txt');
43:Ayuda('ayu.txt');
End;
Accion:=0;
Until Fin;
END;

{///// PROGRAMA PRINCIPAL /////}

```

```

BEGIN
  N_Cargas:=0;          N_Hilos:=0;          Densidad:=8;
  Fac_Esc:=0.02;        Grosor:=0;          Modo_grafico;
  Primera_Pantalla;     Ini_Raton;          Curg;
  Cursor;               Assign(F, 'Color1.cnf'); Reset(F);
  Read(F,M);
  With Pm do
    begin
      Texto:=M.texto;
      Fondo:=M.Fondo;
      Letra:=M.Letra;
      Resalte:=M.Resalte;
    end;
  Close(F);
  Assign(G, 'Color2.cnf');
  Reset(G);
  Read(G,C);
  With Pc do
    begin
      Carga_Mas:=C.Carga_Mas;
      Carga_Menos:=C.Carga_Menos;
      Linea_Mas:=C.Linea_Mas;
      Linea_Menos:=C.Linea_Menos;
    end;
  Close(G);
  With Pull do
    begin
      Ini(0,0,1,True);
      Ini_Color(Pm);
      for i:=1 to 4 do Otro_Item(NP[i],LP[i]);
      Despliegue_XY(x1,y1);      Resaltar_Op(2,True);
      Despliegue_XY(x2,y2);      Resaltar_Op(3,True);
      Despliegue_XY(x3,y3);      Resaltar_Op(4,True);
      Despliegue_XY(x4,y4);      Resaltar_Op(1,True);
      Ver;
    end;
  with Down[1] do
    begin
      Ini(x1,y1,5,False);
      Ini_Color(Pm);
      for i:=1 to 9 do Otro_Item(ND1[i],LD1[i]);
    end;
  with Down[2] do
    begin
      Ini(x2,y2,20,False);
      Ini_Color(Pm);
      for i:=1 to 3 do Otro_Item(ND2[i],LD2[i]);
    end;
  with Down[3] do
    begin
      Ini(x3,y3,30,False);
      Ini_Color(Pm);
      for i:=1 to 8 do Otro_Item(ND3[i],LD3[i]);
    end;
  with Down[4] do
    begin
      Ini(x4,y4,40,False);
      Ini_Color(Pm);
      for i:=1 to 4 do Otro_Item(ND4[i],LD4[i]);
    end;
  Borrar_Pantalla;
  Menu;
  Guardar_Configuracion;
  CloseGraph;

```

END.

{////////// FIN DEL PROGRAMA //////////}

```
{//////////////////////////////////////
///
///      Unidad Gráfico desarrollada por Javier Blanes Bravo      ///
///      para el proyecto "Desarrollo de un Programa Informático  ///
///      como Medio Didáctico en la Enseñanza de la              ///
///      Teoría de Campos"                                         ///
///      SEP-94                                                    ///
///                                                                ///
//////////////////////////////////////
///                                                                ///
///      La TPU incluye 4 procedimientos para                      ///
///      el manejo del de la pantalla en modo gráfico del        ///
///      del Turbo Pascal                                          ///
///                                                                ///
//////////////////////////////////////}
```

UNIT Grafico;

INTERFACE

USES graph;

```
    PROCEDURE Modo_Grafico;
    PROCEDURE Region_Color(X1,Y1,X2,Y2:Word;Color:Byte);
    PROCEDURE Salva_Region(X1,Y1,X2,Y2:Word;
        VAR Talla:Word;VAR P:Pointer);
    PROCEDURE Restaura_Region(X,Y:Word;Talla:Word;VAR P:Pointer);
```

IMPLEMENTATION

```
PROCEDURE Modo_Grafico;
VAR
    Driver,Mode:Integer;
BEGIN
    Driver:=Detect;
    Initgraph(Driver,Mode,'c:\tp\bgi');
    if GraphResult<>grOk then halt(1);
END;
```

```
PROCEDURE Region_Color(X1,Y1,X2,Y2:Word;Color:Byte);
BEGIN
    SetFillStyle(1,Color);
    Bar(X1,Y1,X2,Y2);
END;
```

```
PROCEDURE Salva_Region(X1,Y1,X2,Y2:Word;VAR Talla:Word;VAR P:Pointer);
BEGIN
    Talla:=ImageSize(X1,Y1,X2,Y2);
    System.GetMem(P,(Talla+15)and $fff0);
    GetImage(X1,Y1,X2,Y2,P^);
END;
```

```
PROCEDURE Restaura_Region(X,Y:Word;Talla:Word;VAR P:Pointer);
BEGIN
    PutImage(X,Y,P^,NormalPut);
    System.FreeMem(P,(Talla+15)and $fff0);
```

```
{////////// FIN DE LA TPU GRAFICO  //////////}
```

end;

```
{////////// OBJETO CAJA //////////}
```

```
Caja=Object
```

```
    Cor_1,Cor_2:Cor;  
    Relieve:Byte;  
    Cor_X,Cor_Y:Word;  
    P:Pointer;  
    Guardada:Boolean;  
  
    PROCEDURE Ini_Caja(X1,Y1,X2,Y2:Word;Rel:Byte);  
    PROCEDURE Ver;  
    PROCEDURE Borrar;  
    PROCEDURE Marca(X,Y:Word);  
    PROCEDURE No_Marca(X,Y:Word);  
    PROCEDURE Ver_Paleta(X,Y:Word);  
    PROCEDURE Ver_Posicion_Raton;  
    PROCEDURE Actualizar_Raton;
```

```
end;
```

```
{////////// OBJETO DISPLAY //////////}
```

```
Display=Object
```

```
    Origen:Cor;  
    V_Display:Integer;  
  
    PROCEDURE Ini_Display(X,Y:Word;Val:Integer);  
    PROCEDURE Borrar;  
    PROCEDURE Incremento(Val:Integer);
```

```
end;
```

```
{////////// OBJETO BOTON //////////}
```

```
Boton=Object
```

```
    Cor_1,Cor_2:Cor;  
    Relieve,CTexto:Byte;  
    Texto:String;  
    P:Pointer;  
    Pulsado,Guardada:Boolean;  
  
    PROCEDURE Ini_Boton(X1,Y1,X2,Y2:Word;Text:String;  
        Rel,CText:Byte);  
    PROCEDURE Ver;  
    PROCEDURE Ver_Pulsado;  
    PROCEDURE Borrar;  
    FUNCTION Esta_Pulsado:Boolean;
```

```
end;
```

```
IMPLEMENTATION
```

```
{////////// IMPLEMENTACION OBJETO VENTANA //////////}
```

```
PROCEDURE Ventana.Ini(X,Y,Op_1:Word;Pull:Boolean);
```

```
BEGIN
```

```
    Origen.X:=X;  
    Origen.Y:=Y;  
    Opcion:=1;  
    Numero_Op:=0;  
    Primera_Op:=Op_1;  
    Es_Pull:=Pull;  
    Guardada:=False;  
    if Es_Pull then  
        begin  
            Base:=GetMaxX-X;
```

```

                Altura:=15;
                Sombra:=0;
            end
        else
            begin
                Base:=0;
                Altura:=0;
                Sombra:=10;
            end;
        end;
    END;

    PROCEDURE Ventana.Ini_Color(Col:Paleta_Menu);
    BEGIN
        Colores:=Col;
    END;

    FUNCTION Ventana.Obtener_Op:Byte;
    BEGIN
        Obtener_Op:=Primera_Op+Opcion-1;
    END;

    PROCEDURE Ventana.Resaltar_Op(Op:Byte;Fondo:Boolean);
    VAR
        T:Nodo;
        X1,Y1,X2,Y2:Word;
        R,Espacio:String;
    BEGIN
        Opcion:=Op;
        Espacio:=' ';
        if Fondo then SetFillStyle(1,Colores.Fondo)
            else SetFillStyle(1,Colores.Resalte);
        T:=Tabla[Op];
        X1:=T.Origin.X;
        Y1:=T.Origin.Y;
        if Es_Pull then X2:=X1+(Length(T.Texto)*8+20)
            else X2:=X1+Base-20;
        Y2:=Y1+15;
        Bar(X1,Y1,X2,Y2);
        SetViewPort(X1,Y1,X2,Y2,True);
        SetColor(Colores.Texto);
        T.Texto[Pos(T.Letra,T.Texto)]:=' ';
        OutTextXY(10,4,T.Texto);
        FillChar(R,SizeOf(T.Texto),' ');
        R[Pos(espacio,T.Texto)]:=T.Letra;
        SetColor(Colores.Letra);
        OutTextXY(10,4,R);
        SetViewPort(0,0,GetMaxX,GetMaxY,True);
    END;

    PROCEDURE Ventana.Despliegue_XY(VAR X,Y:Word);
    BEGIN
        X:=Tabla[Opcion].Origin.X;
        Y:=Tabla[Opcion].Origin.Y+16;
    END;

    PROCEDURE Ventana.Otro_Item(T:String;Ch:char);
    VAR
        Longitud:Word;
    BEGIN
        Numero_Op:=Numero_Op+1;
        Tabla[Numero_Op].Texto:=T;
        Tabla[Numero_Op].Letra:=Ch;
        longitud:=(length(T)*10)+20;
        if Es_Pull

```

```

    then
        begin
            Tabla[Numero_Op].Origen:=Origen;
            if Numero_Op<>1
            then
                begin
                    Tabla[Numero_Op].Origen.X:=
                    Tabla[Numero_Op-1].Origen.X+
                    Length(Tabla[Numero_Op-1].Texto)*8+20;
                end;
            end
        else
            begin
                Tabla[Numero_Op].Origen.X:=Origen.X+10;
                Tabla[Numero_Op].Origen.Y:=Origen.Y+(Numero_Op*15)-5;
                if (Longitud>Base) then Base:=Longitud;
                Altura:=(Numero_Op*15)+20;
            end;
        END;

PROCEDURE Ventana.Ver;
VAR
    I,Op:Byte;
    Talla,X,Y:Word;
BEGIN
    X:=Origen.X;
    Y:=Origen.Y;
    Talla:=ImageSize(0,0,Base+Sombra,Altura+Sombra);
    No_Cursor;
    if not(Guardada) then
        begin
            GetMem(P,Talla);
            GetImage(X,Y,X+Base+Sombra,Y+Altura+Sombra,P^);
            Guardada:=True;
        end;
    if Es_Pull then
        begin
            SetFillStyle(2,Colores.Fondo);
            Bar(X,Y,X+Base,Y+Altura);
        end
    else
        begin
            SetFillStyle(1,Colores.Fondo);
            Bar(X,Y,X+Base,Y+Altura);
            SetFillStyle(1,Black);
            Bar(X+10,Y+Altura+1,X+Base+Sombra,Y+Altura+Sombra);
            Bar(X+Base+1,Y+10,X+Base+Sombra,Y+Altura);
            SetColor(Colores.Texto);
            Rectangle(X+9,Y+9,X+Base-9,Y+Altura-9);
        end;
    Op:=Opcion;
    for I:=1 to Numero_Op do Resaltar_Op(I,True);
    Resaltar_Op(Op,False);
    Cursor;
END;

PROCEDURE Ventana.Borrar;
VAR
    Talla:word;
BEGIN
    if guardada then
        Begin
            Talla:=ImageSize(0,0,Base+Sombra,Altura+Sombra);
            No_Cursor;

```

```

        PutImage(Origen.X,Origen.Y,P^,NormalPut);
        Cursor;
        FreeMem(P,Talla);
        Guardada:=False;
    end;
END;

FUNCTION Ventana.Tratar_Evento:Integer;
VAR
    CH:Char;
    A,I:Byte;
    X,Y:Word;
    Exito:Boolean;

FUNCTION Opcion_Raton:Byte;
VAR
    I,X_Raton:Word;
    Exito:Boolean;
BEGIN
    if Es_Pull
    then
        begin
            X_Raton:=Donde_X;
            I:=1;
            Exito:=False;
            while (I<=Numero_Op)and not(Exito) do
                if Tabla[I].Origen.X>X_Raton then Exito:=True
                    else I:=I+1;
            end
            Opcion_Raton:=I-1;
        end
    else Opcion_Raton:=((Donde_Y-Y-10)div 15)+1;
END;

BEGIN
    Tratar_Evento:=0;
    if KeyPressed
    then
        begin
            Ch:=ReadKey;
            if (Ch<>#0)
            then
                begin
                    if Ord(Ch)=13 then Tratar_Evento:=13;
                    if Ord(Ch)=27 then Tratar_Evento:=27;
                    Exito:=false;
                    i:=1;
                    Repeat
                        Exito:=Tabla[i].Letra=Ch;
                        i:=i+1;
                    Until (Exito)or(Numero_Op<i);
                    if Exito
                    then
                        begin
                            No_Cursor;
                            Resaltar_Op(Opcion,True);
                            Opcion:=i-1;
                            Resaltar_Op(Opcion,false);
                            Cursor;
                            Tratar_Evento:=13;
                        end;
                    end
                end
            else
                begin
                    Ch:=readkey;

```



```

Case Ord(Ch) of
72:begin
    if not(Es_Pull) then
        begin
            No_cursor;
            Resaltar_Op(Opcion,True);
            Opcion:=Opcion-1;
            if Opcion=0
                then Opcion:=Numero_Op;
            Resaltar_Op(Opcion,False);
            Cursor;
        end;
    end;
80:begin
    if not(Es_Pull) then
        begin
            No_cursor;
            Resaltar_Op(Opcion,True);
            Opcion:=Opcion+1;
            if Opcion>Numero_op
                then Opcion:=1;
            Resaltar_Op(Opcion,False);
            Cursor;
        end;
    end;
77:begin
    if not(Es_Pull)
    then Tratar_Evento:=1
    else
        begin
            No_cursor;
            Resaltar_Op(Opcion,True);
            Opcion:=Opcion+1;
            if Opcion>Numero_Op
                then Opcion:=1;
            Resaltar_Op(Opcion,False);
            Cursor;
        end;
    end;
75:begin
    if not(Es_Pull)
    then Tratar_Evento:=-1
    else
        begin
            No_cursor;
            Resaltar_Op(Opcion,True);
            Opcion:=Opcion-1;
            if Opcion=0
                then Opcion:=Numero_Op;
            Resaltar_Op(Opcion,False);
            Cursor;
        end;
    end;
end;
end;
end;
end
else
begin
    A:=botones;
    Delay(200);
    if A=1 then
        begin
            X:=Origen.X;
            Y:=Origen.Y;

```

```

        if not(Dentro_Region(X,Y,X+Base,Y+Altura))
        then Tratar_Evento:=27
        else if Dentro_Region(X+Sombra,Y+Sombra,
            X+Base-Sombra,Y+Altura-Sombra)
            then
                begin
                    No_Cursor;
                    Resaltar_Op(Opcion,True);
                    Opcion:=Opcion_Raton;
                    Resaltar_Op(Opcion,False);
                    Cursor;
                    Tratar_evento:=13;
                end;
            end
        else Tratar_Evento:=27;
    end;
END;

{////////// IMPLEMENTACION OBJETO CAJA //////////}

PROCEDURE Caja.Ini_Caja(X1,Y1,X2,Y2:Word;Rel:Byte);
BEGIN
    Cor_1.X:=X1;Cor_1.Y:=Y1;
    Cor_2.X:=X2;Cor_2.Y:=Y2;
    Relieve:=Rel;
    Guardada:=False;
END;

PROCEDURE Caja.Ver;
VAR
    Temp,i:Byte;
    Talla:Word;
BEGIN
    Temp:=GetColor;
    No_Cursor;
    if not(guardada)then
        begin
            Talla:=ImageSize(Cor_1.X,Cor_1.Y,Cor_2.X,Cor_2.Y);
            GetMem(P,Talla);
            GetImage(Cor_1.X,Cor_1.Y,Cor_2.X,Cor_2.Y,P^);
            Guardada:=True;
        end;
    SetFillStyle(1,LightGray);
    Bar(Cor_1.X,Cor_1.Y,Cor_2.X,Cor_2.Y);
    SetColor(Black);
    Line(Cor_1.X,Cor_2.Y,Cor_2.X,Cor_2.Y);
    Line(Cor_2.X,Cor_2.Y,Cor_2.X,Cor_1.Y);
    for i:=0 to Relieve-1 do
        Rectangle(Cor_1.X+5+i,Cor_1.Y+5+i,Cor_2.X-5-i,Cor_2.Y-5-i);
    SetColor(White);
    Line(Cor_1.X,Cor_2.Y,Cor_1.X,Cor_1.Y);
    Line(Cor_1.X,Cor_1.Y,Cor_2.X,Cor_1.Y);
    for i:=0 to Relieve-1 do
        begin
            Line(Cor_1.X+5+i,Cor_2.Y-5-i,Cor_2.X-5-i,Cor_2.Y-5-i);
            Line(Cor_2.X-5-i,Cor_2.Y-5-i,Cor_2.X-5-i,Cor_1.Y+5+i);
        end;
    Cursor;
END;

PROCEDURE Caja.Borrar;
VAR
    Talla:Word;
BEGIN

```

```

if Guardada then
begin
    Talla:=ImageSize(Cor_1.X,Cor_1.Y,Cor_2.X,Cor_2.Y);
    No_Cursor;
    PutImage(Cor_1.X,Cor_1.Y,P^,NormalPut);
    Cursor;
    FreeMem(P,Talla);
    Guardada:=False;
end;
END;

PROCEDURE Caja.Marca(X,Y:Word);
VAR
    I:Byte;
BEGIN
    No_Cursor;
    SetColor(Red);
    SetTextStyle(0,0,1);
    OutTextxy(X,Y,'û');
    SetTextStyle(0,0,0);
    Cursor;
END;

PROCEDURE Caja.No_Marca(X,Y:Word);
VAR
    I:Byte;
BEGIN
    No_Cursor;
    SetColor(LightGray);
    SetTextStyle(0,0,1);
    OutTextXY(X,Y,'û');
    SetTextStyle(0,0,0);
    Cursor;
END;

PROCEDURE Caja.Ver_Paleta(X,Y:Word);
VAR
    I:Byte;
BEGIN
    No_Cursor;
    SetColor(White);
    for i:=0 to 2 do
        Rectangle(X+i,Y+i,X+246-i,Y+21-i);
    SetColor(DarkGray);
    for i:=0 to 2 do
        begin
            Line(X+i,Y+21-i,X+246-i,Y+21-i);
            Line(X+246-i,Y+21-i,X+246-i,Y+i);
        end;
    for I:=0 to 15 do
        begin
            SetFillStyle(1,I);
            Bar(X+3+(I*15),Y+3,X+18+(I*15),Y+18);
        end;
    Cursor;
END;

PROCEDURE Caja.Ver_Posicion_Raton;
BEGIN
    SetColor(LightRed);
    OutTextXY(Cor_1.X+20,Cor_1.Y+10,'COORD X:');
    OutTextXY(Cor_1.X+20,Cor_1.Y+20,'COORD Y:');
END;

```

```

PROCEDURE Caja.Actualizar_Raton;
VAR
  T1,T2:String;
  X,Y:Word;
BEGIN
  X:=Donde_X;
  Y:=Donde_Y;
  Str(Cor_X,T1);
  Str(Cor_Y,T2);
  Cor_X:=X;
  Cor_Y:=Y;
  SetColor(LightGray);
  OutTextXY(Cor_1.X+90,Cor_1.Y+10 ,T1);
  OutTextXY(Cor_1.X+90,Cor_1.Y+20 ,T2);
  Str(X,T1);
  Str(Y,T2);
  SetColor(1);
  OutTextXY(Cor_1.X+90,Cor_1.Y+10 ,T1);
  OutTextXY(Cor_1.X+90,Cor_1.Y+20 ,T2);
END;

{////////// IMPLEMENTACION OBJETO DISPLAY //////////}

PROCEDURE Display.Ini_Display(X,Y:Word;Val:Integer);
VAR
  T:String;
BEGIN
  Origen.X:=X;
  Origen.Y:=Y;
  SetColor(0);
  No_Cursor;
  Rectangle(X,Y,X+60,y+20);Rectangle(X+1,Y+1,X+59,Y+19);
  V_Display:=Val;
  Str(Val,T);
  SetColor(1);
  OutTextXY(X+5,Y+4,T);
  Cursor;
END;

PROCEDURE Display.Borrar;
BEGIN
  No_Cursor;
  SetFillStyle(1,LightGray);
  Bar(Origen.X,Origen.Y,Origen.X+40,Origen.Y+20);
  Cursor;
END;

PROCEDURE Display.Incremento(Val:Integer);
VAR
  T:String;
BEGIN
  Str(V_display,T);
  SetColor(LightGray);
  No_Cursor;
  OutTextXY(Origen.X+5,Origen.Y+4,T);
  V_Display:=V_Display+Val;
  Str(V_Display,T);
  SetColor(1);
  OutTextXY(Origen.X+5,Origen.Y+4,T);
  Cursor;
END;

{////////// IMPLEMENTACION OBJETO BOTON //////////}

```

```

PROCEDURE Boton.Ini_Boton(X1,Y1,X2,Y2:Word;Text:String;Rel,CText:Byte);
BEGIN
    Pulsado:=False;
    Cor_1.X:=X1;Cor_1.Y:=Y1;
    Cor_2.X:=X2;Cor_2.Y:=Y2;
    Texto:=Text;
    Relieve:=Rel;
    CTexto:=CText;
END;

```

```

PROCEDURE Boton.Ver;
VAR
    I:Byte;
BEGIN
    Pulsado:=False;
    No_Cursor;
    SetColor(Blue);
    Rectangle(Cor_1.X,Cor_1.Y,Cor_2.X,Cor_2.Y);
    SetColor(White);
    for i:=1 to Relieve do
        Rectangle(Cor_1.X+i,Cor_1.Y+i,Cor_2.X-i,Cor_2.Y-i);
    SetColor(DarkGray);
    for i:=1 to Relieve do
        begin
            Line(Cor_1.X+i,Cor_2.Y-i,Cor_2.X-i,Cor_2.Y-i);
            Line(Cor_2.X-i,Cor_2.Y-i,Cor_2.X-i,Cor_1.Y+i);
        end;
    SetFillStyle(1,LightGray);
    Bar(Cor_1.X+Relieve+1,Cor_1.Y+Relieve+1,
        Cor_2.X-Relieve-1,Cor_2.Y-Relieve-1);
    SetColor(CTexto);
    OutTextXY(Cor_1.X+Relieve+3,Cor_1.Y+8,Texto);
    Cursor;
END;

```

```

PROCEDURE Boton.Ver_Pulsado;
BEGIN
    Pulsado:=True;
    No_Cursor;
    SetColor(Blue);
    Rectangle(Cor_1.X,Cor_1.Y,Cor_2.X,Cor_2.Y);
    SetFillStyle(1,Black);
    Bar(Cor_1.X+1,Cor_1.Y+1,Cor_2.X-1,Cor_2.Y-1);
    SetFillStyle(1,LightGray);
    Bar(Cor_1.X+3,Cor_1.Y+3,Cor_2.X-1,Cor_2.Y-1);
    SetColor(Black);
    OutTextXY(Cor_1.X+Relieve+3,Cor_1.Y+8,Texto);
    Cursor;
END;

```

```

PROCEDURE Boton.Borrar;
BEGIN
    No_Cursor;
    SetFillStyle(1,LightGray);
    Bar(Cor_1.X,Cor_1.Y,Cor_2.X+1,Cor_2.Y);
    Cursor;
END;

```

```

FUNCTION Boton.Esta_Pulsado:Boolean;
BEGIN
    Esta_Pulsado:=(Pulsado)or((Dentro_Region(Cor_1.X+Relieve,
        Cor_1.Y+Relieve,Cor_2.X-Relieve,Cor_2.Y-Relieve))
        and (botones<>0));
END;

```

END.

{////////// FIN DE LA TPU //////////}

```
{//////////////////////////////////////
///
///      Unidad Pantalla desarrollada por Javier Blanes Bravo      ///
///      para el proyecto "Desarrollo de un Programa Informático    ///
///      como Medio Didáctico en la Enseñanza de la                ///
///      Teoría de Campos"                                          ///
///      SEP-94                                                      ///
///      //////////////////////////////////////////////////////////////////////////
///
///      La TPU incluye 3 procedimientos y funciones para          ///
///      el manejo de la pantalla                                   ///
///      //////////////////////////////////////////////////////////////////////////}
```

UNIT Pantalla;

INTERFACE

USES Dos,Crt,Cargas,Graph;

PROCEDURE Dibuja_Pto(P:Pto;Color,Grosor:Byte);

FUNCTION Fuera_Pan(P:Pto):Boolean;

PROCEDURE Texto(X,Y,Color:Word;T:String);

IMPLEMENTATION

PROCEDURE Dibuja_Pto(P:Pto;Color,Grosor:Byte);

VAR

X,Y:Word;

I,J:Byte;

BEGIN

X:=Round(P.X*100);

Y:=Round(P.Y*100);

For I:=0 to Grosor do

For J:=0 to Grosor do

PutPixel(X+I,Y+J,Color);

END;

function Fuera_Pan(P:Pto):Boolean;

BEGIN

Fuera_Pan:=False;

if (P.X>6.39) or (P.Y>4.79) or (P.X<0) or (P.Y<0.15)

then Fuera_Pan:=True;

END;

procedure Texto(X,Y,Color:Word;T:String);

BEGIN

SetTextJustify(CenterText,CenterText);

SetColor(White);

OutTextXY(X+1,Y+2,T);

SetColor(Color);

OutTextXY(X,Y,T);

SetTextJustify(0,2);

END;

END.

{////////// FIN DE LA TPU PANTALLA //////////}

```

{////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////}
///
///      Unidad Ratón desarrollada por Javier Blanes Bravo      ///
///      para el proyecto "Desarrollo de un Programa Informático  ///
///      como Medio Didáctico en la Enseñanza de la            ///
///      Teoría de Campos"                                       ///
///      SEP-94                                                  ///
///
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////}
///
///      La TPU incluye 14 procedimientos y funciones para      ///
///      el manejo del ratón asociados a la interrupción 51    ///
///
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////}

```

UNIT Raton;

INTERFACE

USES Crt,Dos;

```

PROCEDURE Ini_Raton;
PROCEDURE Cursor;
PROCEDURE No_Cursor;
FUNCTION  Ini_Ok:Boolean;
FUNCTION  Numero_Botones:Byte;
FUNCTION  Cursor_Visible:Boolean;
FUNCTION  Donde_X:Word;
FUNCTION  Donde_Y:Word;
FUNCTION  Botones:Byte;
FUNCTION  Dentro_Región(X1,Y1,X2,Y2:Word):Boolean;
PROCEDURE Limites(X1,Y1,X2,Y2:Word);
PROCEDURE Area_Exclusion(X1,Y1,X2,Y2:Word);
PROCEDURE Ve_A(X,Y:word);
PROCEDURE CurG;

```

IMPLEMENTATION

{////////// IMPLEMENTACION DE LOS PROCEDIMIENTOS Y FUNCIONES //////////}

VAR

```

R1,R2,R3,R4,R5,R6,R7:Word;
Visible,Ini_Bien:Boolean;
NBotones:Word;

```

PROCEDURE Interrupcion_51;

{///// Procedimiento a bajo nivel que llama a la interrupción 51 /////}

VAR

Reg:Registers;

BEGIN

```

with Reg do
begin

```

```

    ax:=R1;
    bx:=R2;
    cx:=R3;
    dx:=R4;
    si:=R5;
    di:=R6;
    es:=R7;

```

```

        end;
    Intr(51,Reg);
    with reg do
        begin
            R1:=ax;
            R2:=bx;
            R3:=cx;
            R4:=dx;
        end;
    END;

PROCEDURE Ini_Raton;
BEGIN
    R1:=0;
    Interrupcion_51;
    if R1=-1 then Ini_Bien:=True
        else Ini_Bien:=False;
    NBotones:=R2;
END;

PROCEDURE Cursor;
BEGIN
    R1:=1;
    Interrupcion_51;
    Visible:=True;
END;

PROCEDURE No_Cursor;
BEGIN
    R1:=2;
    Interrupcion_51;
    Visible:=False;
END;

FUNCTION Ini_Ok:Boolean;
BEGIN
    Ini_Ok:=Ini_Bien;
END;

FUNCTION Numero_Botones:Byte;
BEGIN
    Numero_Botones:=NBotones;
END;

FUNCTION Cursor_Visible:Boolean;
BEGIN
    Cursor_Visible:=Visible;
END;

FUNCTION Donde_X:Word;
BEGIN
    R1:=3;
    Interrupcion_51;
    Donde_X:=R3;
END;

FUNCTION Donde_Y:Word;
BEGIN
    R1:=3;
    Interrupcion_51;
    Donde_Y:=R4;
END;

FUNCTION Botones:Byte;

```



```

BEGIN
    R1:=3;
    Interrupcion_51;
    Botones:=R2;
END;

FUNCTION Dentro_Region(X1,Y1,X2,Y2:Word):Boolean;
VAR
    X,Y:Word;
BEGIN
    X:=Donde_X;
    Y:=Donde_Y;
    Dentro_Region:=(X>=X1)and(Y>=Y1)and(X<=X2)and(Y<Y2);
END;

PROCEDURE Limites(X1,Y1,X2,Y2:Word);
BEGIN
    R1:=7;R3:=X1;R4:=X2;
    Interrupcion_51;
    R1:=8;R3:=Y1;R4:=Y2;
    Interrupcion_51;
END;

PROCEDURE Area_Exclusion(X1,Y1,X2,Y2:Word);
BEGIN
    R1:=10;
    R3:=x1;
    R4:=y1;
    R5:=x2;
    R6:=y2;
    Interrupcion_51;
END;

PROCEDURE Ve_A(X,Y:word);
BEGIN
    R1:=4;
    R2:=X;
    R3:=Y;
    Interrupcion_51;
END;

PROCEDURE CurG;
var V:array[1..32]of word;
    i:Byte;
BEGIN
    v[1]:=57855; v[2]:=57855; v[3]:=57855; v[4]:=57855; v[5]:=57855;
    v[6]:=57344; v[7]:=57344; v[8]:=0; v[9]:=0; v[10]:=0;
    v[11]:=0; v[12]:=0; v[13]:=0; v[14]:=32769;v[15]:=49155;
    v[16]:=57351;v[17]:=7680; v[18]:=4608; v[19]:=4608; v[20]:=4608;
    v[21]:=4608; v[22]:=5119; v[23]:=4681; v[24]:=62025;v[25]:=37449;
    v[26]:=37449;v[27]:=32769;v[28]:=32769;v[29]:=32769;v[30]:=16386;
    v[31]:=8196; v[32]:=8184;
    R1:=9;
    R2:=2;
    R3:=2;
    R4:=ofs(v);
    R7:=Seg(v);
    Interrupcion_51;
END;

BEGIN
    Visible:=false;
    Ini_Bien:=false;
    NBotones:=0;

```

```
R1:=0;R2:=0;R3:=0;R4:=0;R5:=0;R6:=0;
END.
```

```
{////////// FIN DE LA TPU RATON //////////}
```

```
{//////////////////////////////////////
///
///      Unidad Sonido desarrollada por Javier Blanes Bravo      ///
///      para el proyecto "Desarrollo de un Programa Informático  ///
///      como Medio Didáctico en la Enseñanza de la              ///
///      Teoría de Campos"                                         ///
///      SEP-94                                                    ///
///                                                                ///
//////////////////////////////////////
///
///      La TPU incluye 7 procedimientos de efectos sonoros      ///
///                                                                ///
//////////////////////////////////////}
```

```
UNIT Sonido;
```

```
INTERFACE
```

```
USES DOS,CRT;
```

```
PROCEDURE Pita1(n:Word) ;
PROCEDURE Pita2(n:Word) ;
PROCEDURE Pita3(n:Word) ;
PROCEDURE Bip1 ;
PROCEDURE Bip2 ;
PROCEDURE Sirena(Fre:Word);
PROCEDURE Sirena_Larga(Fre:Word) ;
```

```
IMPLEMENTATION
```

```
PROCEDURE Pita1(n:Word) ;
BEGIN
    Sound(260);
    Delay(n);
    NoSound;
END;
```

```
PROCEDURE Pita2(n:Word) ;
BEGIN
    Sound(190);
    Delay(n);
    NoSound;
END;
```

```
PROCEDURE Pita3(n:Word) ;
BEGIN
    Sound(120);
    delay(n);
    NoSound;
END;
```

```
PROCEDURE Bip1 ;
Var i : Integer ;
BEGIN
    For i:=1 To 2 Do
        BEGIN
```

```

                Sound(200);
                delay(40);
                NoSound;
                delay(60);
                Sound(160);
                delay(40);
                NoSound;
            END;
        END;

PROCEDURE Bip2 ;
Var i : Integer ;
BEGIN
    For i:=1 To 2 Do
        BEGIN
            Sound(1200);
            delay(40);
            NoSound;
            delay(60);
            Sound(900);
            delay(40);
            NoSound;
        END;
    END;

PROCEDURE Sirena(Fre:Word);

    BEGIN
        If (Fre>100) Then Sirena(fre-100);
        Sound(fre);
        Delay(10);
        NoSound;
    END;

PROCEDURE Sirena_Larga(Fre:Word);
BEGIN
    If (Fre>100) Then Sirena(fre-100);
    Sound(fre);
    Delay(60);
    NoSound;
END;
END.

{////////// FIN DE LA TPU SONIDO //////////}

```

```

*****
*** AYUDA DE LA OPCION ***
*** AYUDA DEL MENU ***
*****

```

La \$Ayuda\$ se maneja con las teclas \$ENTER\$ para pasar de página y \$Esc\$ para terminar.

```

*****
*** AYUDA DE LA OPCION ***
*** COLORES DEL MENU ***
*****

```

Se deberá establecer el color del epígrafe seleccionado, correspondiendo cada uno a:

El \$fondo\$ es sobre el que se escribe el \$texto\$ del menú. \$Letra\$ simboliza el color de la tecla rápida y \$resalte\$ el color con el cual se resalta la opción. \$Carga_+/- \$ se refiere al color con el cual se representan las cargas y \$Línea_+/- \$ de las líneas de campo.

```
*****
*** AYUDA DE LA OPCION ***
*** OPCIONES DEL MENU ***
*****
```

\$-Lineas de Campo:\$ Dibuja las líneas de campo de los elementos hilo , carga que existan configurados actualmente.

\$-Superficies Equipotenciales: \$Dibuja las superficies equipotenciales de los elementos hilo, carga que existan configurados actualmente.

\$-Situación Cargas:\$ Después de pulsar esta opción, deberá situar con ayuda del ratón la carga en el sitio deseado y oprimir la opción \$ENTER\$ del ratón, tras lo cual aparecerá un cuadro de diálogo que deberá establecer correctamente en cuanto a valor de la carga, oprimir \$Ok!\$ cuando el valor consignado sea el deseado, y oprimir

\$Otra\$ si desea repetir la operación o \$Fin\$ si desea concluir.

\$-Situación Hilos:\$ Se procederá de manera análoga al caso anterior, pero se deberá tener en cuenta que:

- Después de situar el extremo superior-izquierdo del hilo, se contestará mediante teclado a la pregunta Horizontal Vertical?
- Y después se situará el otro extremo, procediéndose luego como en el caso anterior.

\$-Borrar Cargas:\$ Permite borrar cargas simplemente pulsando \$ENTER\$ del ratón sobre la carga deseada y el proceso finaliza pulsando el \$ESCAPE\$ del ratón.

\$-Borrar Hilos:\$ Idem al caso anterior, pero con los elementos hilo.

\$-Modificar Cargas:\$ Permite modificar cargas simplemente pulsando \$ENTER\$ del ratón sobre la carga deseada y posteriormente redefiniendo su valor.

\$-Modificar Hilos:\$ Idem al caso anterior, pero con los elementos hilo.

\$-Fin:\$ Si se pulsa esta opción se sale del programa.

```
*****
*** AYUDA DE LA OPCION ***
*** PARAMETROS DEL MENU ***
*****
```

\$-Densidad de Líneas:\$ Se deberá establecer el valor del cuadro de diálogo que simboliza el número de líneas que saldrán de cada elemento carga o hilo.

\$-Rapidez:\$ Se deberá establecer el valor del cuadro de diálogo que simboliza la rapidez-precisión del trazado de las líneas de campo.

\$-Grosor de Línea:\$ Se deberá establecer a 1 o a 0 este valor y simboliza lo grueso que será el trazado de las líneas de campo y de las superficies equipotenciales.